

FIDELITYLOOP: CLOSED-LOOP VALIDATION OF ELASTIC LLM-SERVING POLICIES

Anonymous Authors¹

ABSTRACT

Elastic LLM-serving policies depend on when capacity becomes available, not just on request service times. FidelityLoop evaluates these policies by connecting replay predictions to physical lifecycle execution and deadline-aware resource accounting. We ask whether a policy selected in replay still meets its deadlines when deployed. In a preregistered study of 36 Qwen2.5-7B-Instruct runs on two A100 GPUs, service calibration improves matched-request timing while worsening occupied-resource prediction; trajectory analysis explains why aggregate error is insufficient for closed-loop evaluation. We then execute five frozen PPO checkpoints and an all2 capacity anchor in a continuous 32-run campaign on the same registered serving stack, using a separately re-provisioned runtime lock. Twenty runs meet every online and offline deadline: three checkpoints pass all six runs and two fail all six. Both training environments produce passing and failing checkpoints. The service-calibrated replay with the original startup endpoint (C) predicts feasibility correctly on 32/32 runs, with 0.0804 scenario-USD cost MAE. Changing startup to 30 seconds (C30) improves the earlier resource metric but yields only 20/32 correct feasibility predictions and 0.4657 cost MAE. These frozen predictions show why policy evaluation must check deadline feasibility alongside resource accuracy. The artifact retains every attempt and negative result under the studied workloads, hardware, and synthetic-API cost assumptions.

1 INTRODUCTION

Elastic LLM serving releases idle instances to save resources, but capacity returns only after startup. Queues, running work, and lifecycle transitions then shape subsequent control decisions. A simulator used to evaluate such policies must capture when capacity is available, not just how long individual requests take or how many GPU seconds a run consumes.

FidelityLoop studies these requirements as a closed loop from prediction to policy decision to physical lifecycle execution. In 36 preregistered runs, service calibration improves matched-request timing yet worsens occupied-resource prediction under the same hysteresis rule. A startup-endpoint contrast locates one feedback path at the controller’s cooldown boundary, while trajectory analysis shows that equal total occupation error can hide different control inputs. A final 32-cell campaign executes five frozen PPO checkpoints and an all2 capacity anchor on the same registered Qwen/A100 serving stack, but under a separately re-provisioned runtime lock. It exposes which predicted

feasible policies remain feasible when real lifecycle and deadline accounting are applied.

The paper makes three contributions:

1. **A closed-loop evaluation framework.** FidelityLoop separates the policy adapter from lifecycle execution and aligns causal observations, request identities, lifecycle generations, resource trajectories, and deadline-aware cost ledgers. The same contract runs fixed rules and learned checkpoints, with failed requests retained in the denominator.
2. **A lifecycle-fidelity diagnosis.** Held-out physical runs distinguish local-service, aggregate-resource, and trajectory fidelity. Event-level comparisons explain a startup-related divergence and show why aggregate resource error is insufficient for a closed-loop evaluator.
3. **A reproducible policy evaluation.** A deadline guard quantifies the resource–latency trade-off, and the continuous E2 campaign tests five frozen PPO checkpoints under two arrival regimes. The resulting negative cases define the boundary of the framework: lower scenario cost is not a saving when deadlines are missed.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

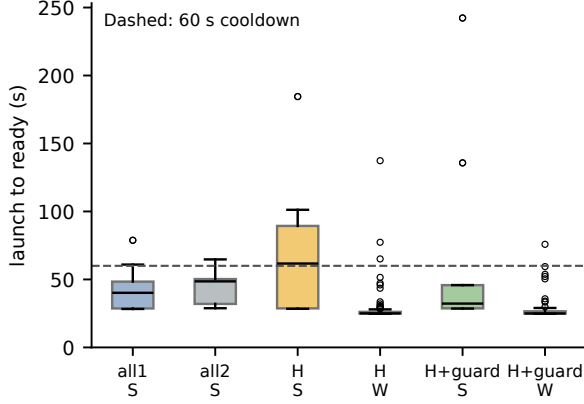


Figure 1. Observed startup times in the held-out runs. Labels use S for setup and W for window scale-out. The dashed line marks the 60-second controller cooldown; boxes summarize observed launch-to-ready times. Release and cleanup are separate ledger events. Collapsing these contexts into one endpoint can alter closed-loop actions.

interface; its scope is reported explicitly rather than treated as a public hardware emulator.

2 BACKGROUND AND MOTIVATION

The exploratory deployment exposed an asymmetry: calibration made matched local service error smaller while dynamic occupied-resource error grew. These inherited values motivate the preregistered study; they are not additional held-out repeats.

The lifecycle diagnosis separated the old 69.219 s startup proxy from the earlier window launch-to-ready (25.19 s) and setup (46.80 s) contexts. When the proxy crossed the 60 s cooldown boundary it changed a target decision, motivating the locked sequence of calibration, lifecycle checks, and held-out feasibility tests.

Figure 1 summarizes the lifecycle boundary. It is a motivating diagnostic, not a post-hoc selection rule: the held-out windows were selected once from the registered forward pool and were not replaced after observing performance.

3 RELATED WORK

Serving predictors support validation and configuration choice: Vidur simulates LLM inference, Charon models operator-level execution, and Clockwork studies predictable request timing (Agrawal et al., 2024; Yang et al., 2026; Gujarati et al., 2020). We ask a narrower validation question: can better matched-service fit coexist with worse launch-to-verified-release prediction under a fixed controller?

Physical serving systems change how capacity becomes

available: cold-start characterization and systems such as ServerlessLLM expose this lifecycle cost (Kabakibo et al., 2026; Fu et al., 2024). We hold the engine and loading path fixed, changing only simulated timing in the endpoint comparison.

Provisioning and scheduling work connects workload estimates to SLOs (Crankshaw et al., 2018; Zhang et al., 2019; 2023). We instead evaluate a visible-state guard for two non-preemptive instances and distinguish setup, scale-out, and release in a closed loop. Our focus is the held-out discrepancy between prediction levels and the startup-endpoint contrast that helps explain it.

Policy design provides a complementary layer. Llum-nix dynamically reschedules LLM requests (Sun et al., 2024); AAPA predicts scaling needs for serverless workloads (Zhang et al., 2025b); KIS-S learns GPU-inference autoscaling with PPO (Zhang et al., 2025a). We use PPO as a frozen policy family rather than proposing a new RL algorithm; PPO is the standard clipped policy-gradient method (Schulman et al., 2017). Our question is whether a learned checkpoint selected in replay remains feasible after the lifecycle backend and deadline ledger are applied.

4 PROBLEM AND METRICS

The system has two accelerators, online arrivals with a 60 s deadline, and 120 offline jobs per 2,100 s observation window with a 300 s deadline. Requests may be served locally or accepted by a synthetic API sink. The primary window cost is

$$C = .001G + .002S + .001D + 10^{-6}I + 3 \times 10^{-6}B + .01M, \quad (1)$$

where G is occupied GPU seconds, S, D are starts and shutdowns, I, B are synthetic API input and output budgets, and M counts offline deadline misses, including unfinished requests whose deadlines have passed. Setup and cleanup are reported separately and are not folded into the primary model gate.

Completed means a recorded completion by the observation cutoff; it may be *on time* (finish time $\leq$ deadline) or *late* (finish time $>$ deadline). An unfinished request has no completed terminal by the cutoff. Unfinished or censored work is retained in the arrival denominator. The capacity gate G_F requires all2 (two GPUs throughout) to complete every online and offline request on time. P1 requires 120/120 offline requests on time, at least 99% online requests on time, and no unfinished request; P1+ additionally requires every online request on time. P2 is a ratio-of-means saving against all1 computed within each window and then averaged equally across the three windows.

Table 1. Selected window arrival shape at 60 s bins. Each cell is raw/derived; CV and peak-to-mean use the full 1,800 s window, including empty bins.

Window	arrivals	CV	peak/mean	lag-1
steady	5461 / 1444	0.378 / 0.412	1.566 / 1.641	0.772 / 0.797
recovery	1141 / 334	1.444 / 1.452	3.550 / 3.862	0.906 / 0.892
burst-offline	663 / 177	1.951 / 1.935	6.652 / 6.780	0.810 / 0.754

5 WORKLOAD CONSTRUCTION

Online arrivals come from the successful-request subset of BurstGPT, a public trace of production LLM requests (Wang et al., 2024). We read all 1,404,294 source rows and retain source hashes and row identities. The trace has no session identifier, so the split separates arrival times rather than sessions. Locally routed requests execute actual GPU inference with fixed-corpus prompts and registered token budgets. The experiment reproduces arrival-time patterns, not the original users’ prompts or semantic tasks.

The three held-out windows are selected once from the forward pool after the old windows plus a 30-minute protection band. A 60 s morphology filter defines the steady, recovery, and burst-offline candidate classes; a fixed SHA ordering takes the first eligible candidate in steady–recovery–burst order, subject to the registered separation constraint. Online rows are thinned with the inherited C2 integer threshold, while the 120 offline jobs are constructed at fixed 15 s spacing and 256-input/256-output token budgets. The two salts and all thresholds are fixed in `s4_generation_lock.json`; generated inputs are locked before replay and never replaced to improve a result.

Table 1 reports the arrival shape at the registered 60 s scale. The 1 s diagnostics are retained in the manifest but do not define the class. The values describe the selected derived windows, not the full source population.

The derived windows preserve arrival timing and fixed work accounting, but they are late-source slices with no session fields. They therefore represent a controlled workload domain for lifecycle decisions, not a claim about all production traffic or user intent.

E2 uses two separately generated and locked Bridge windows from the same construction pipeline, with no row overlap with the three held-out windows; the campaigns remain distinct and are never pooled. The recovery window contains 358 derived online arrivals and 120 fixed offline arrivals (478 total); the steady window contains 944 online and 120 offline arrivals (1,064 total). Offline arrivals are spaced every 15 s, with 256 input and output tokens; the online rows retain their locked arrival timestamps. The window identities and input hashes are bound in the E2 lock.

6 EXPERIMENTAL SETUP

The preregistered held-out study used two NVIDIA A100 80GB PCIe GPUs at a university computing facility, with one independent tensor-parallelism-1 instance per GPU and Qwen2.5-7B-Instruct. The serving stack was vLLM 0.9.0 (Kwon et al., 2023) with PyTorch 2.7.0+cu126, Transformers 4.51.1, tokenizers 0.21.1, and NumPy 1.26.4. Instances used FP16, maximum model length 4096, maximum batched tokens 16,384, maximum sequences 1,024, four local request slots, chunked prefill enabled, prefix caching disabled, eager execution enabled, and GPU memory utilization 0.8. Readiness required the per-process health probe and the dual-ready barrier; cleanup required verified process release on both devices. These values are bound by each run’s environment snapshot and GPU identity record.

The earlier reference campaign used two A800 GPUs. This hardware change is a deployment drift rather than an implementation detail: the drift probe observed the startup end-point changing from the old 69.219 s proxy to a deployment-adapted 33.497 s value, with a 2.224 s normal-shutdown estimate. E was refrozen from this pre-held-out drift probe before the new windows; V remains bound to the earlier environment lock. This timing is why the V-versus-E result is reported with an environment caveat.

The E2 campaign instantiated the same registered Qwen2.5-7B-Instruct serving stack on two A100 GPUs through the framework contract. It contains 32 locked cells: one all2 capacity anchor and five frozen PPO arms per window, with three repeats per comparison arm across recovery and steady windows. The campaign was run under a new runtime lock, with each checkpoint and tensor identity bound into the result package. It is a policy-execution test, not a model-selection search or a claim of universal PPO superiority. The earlier 38-cell Bridge replay is retained as historical supplementary evidence and is not pooled with E2.

7 METHOD

The physical controller separates policy decisions from lifecycle execution and request accounting. Fixed adapters read observable queues and device states and return capacity targets; the guard may also select dispatch priority. The backend handles readiness, draining, and verified release, while request identities align predictions, decisions, lifecycle states, resources, and costs. The resulting data flow is *workload rows* → *causal observation* → *policy adapter* → *lifecycle backend* → *identity ledger*; a CPU replay recomputes feasibility and cost from raw records. The artifact additionally provides a bounded CPU facade with injectable estimators and policy adapters: four historical configurations replay exactly, and two synthetic configurations pass request, state, and cost conservation checks. These are inter-

face checks, not evidence of heterogeneous-GPU scalability; the artifact records the checks in `VALIDATION.json` and `test_facade.py`. The earlier diagnostic experiments and the Bridge campaign use separate locks and are not pooled.

7.1 Calibrated models

The model family predicts service time, startup, shutdown, active and occupied resource use. O is the original model; C retains the calibrated service model but its old startup endpoint; C30 changes that endpoint to 30 s; E combines the calibrated service model with deployment-adapted lifecycle parameters; V performs one bounded closed-loop calibration around E. The lock fixes model bytes, tokenizer, workload identities, and candidate grid before held-out execution. The symbols match the artifact filenames.

For each model prediction and physical repeat of a window, the dimensionless lifecycle loss is

$$L = \frac{0.5|\Delta G_{\text{occupied}}| + 0.25|\Delta G_{\text{active}}|}{2T} + \frac{0.25|\Delta Q|}{TN}, \quad (2)$$

where Δ is prediction minus observation, $T = 2100$ s for formal windows, and N counts *all* online and offline arrivals. Q is the time integral of the released-but-undispatched queue, in request seconds. Unfinished requests remain in N and contribute queue time while queued up to cutoff. Two devices give the $2T$ normalization. We first average losses over repeats within each window, then weight windows equally; we do not pool requests or restrict N to matched-local support. L measures prediction error, separately from scenario cost C ; the shorter development horizon is stated with that experiment.

7.2 PPO policy contract

The learned policy is trained only in the CPU replay environment, with C or C30 as the training environment; the physical serving loop is used only after a checkpoint is frozen. At each one-second decision boundary, its 30-dimensional causal observation contains normalized time and target state, queue counts and ages/slack, device lifecycle states and ages, running-job summaries, and the API queue. The capacity action proposes a target of 0, 1, or 2 GPUs; the registered 60 s cooldown filters that proposal before execution, while the offline action head is fixed to its sole legal value. The reward is the negative incremental scenario-ledger cost, so training trades occupied resources, lifecycle events, API use, and deadline misses under the declared contract.

Training uses the three development arrival regimes and paired seeds from the locked CPU PPO contract. Checkpoints are selected mechanically with C30 replay, guard disabled, P1+ required in every validation window, and sorting by equal-window mean cost, checkpoint index, then

seed. In the arm names, `r2` denotes the second eligible checkpoint under that contract; `s101` is the registered seed-101, episode-40 matched-coordinate pair; and `s103` is the best eligible checkpoint within seed 103. These labels identify frozen checkpoints and do not encode a post-hoc choice from E2 physical outcomes.

7.3 Controller and guard

H uses up-threshold 8, down-threshold 0, and a 60 s cooldown. At each one-second tick, H reads queue and running counts plus its target/cooldown state, not the predictor’s startup parameter. The guard’s lifecycle-delay estimates use separately frozen safety parameters, distinct from the C/E simulator endpoints. The guard observes only released queues, device states, and running-job ages; future arrivals and true completion or readiness times are unavailable. Let P_t be the pending offline request IDs and b_t H’s current target after its decision. The projection $F_j(k, r)$ estimates request j ’s finish with k devices and reservation setting r . It chooses devices by estimated lifecycle delay (device ID breaks ties), creates four slots per device, and includes running residuals. Service durations use the frozen concurrency-four lookup. An overdue running estimate falls back to one positive full-service duration; starting and stopping devices retain lifecycle delay rather than becoming instantaneous capacity.

Offline jobs are ordered by deadline then request ID. Without reservation, visible online backlog precedes them in the list schedule. With reservation and online backlog, offline work is projected serially from the earliest running offline slot, or the earliest slot if none is running; other slots remain online-first. With no online backlog, offline jobs use all projected slots. These are conservative estimates, not proved completion-time upper bounds.

For the full guard ($r = 1$), margin $m = 5$ s and previous risk-ID set A_{t-1} , the update is

$$\begin{aligned} R_t &= \{j \in P_t : F_j(\max(1, b_t), 1) + m + 1 \geq d_j\}, \\ A_t &= (A_{t-1} \cap P_t) \cup R_t, \\ K_t &= \{k \in \{1, 2\} : \forall j \in P_t, F_j(k, 1) + m \leq d_j\}, \\ u_t &= \begin{cases} b_t, & A_t = \emptyset, \\ \max(b_t, \min K_t), & |A_t| > 0, |K_t| > 0, \\ 2, & |A_t| > 0, K_t = \emptyset. \end{cases} \quad (3) \end{aligned}$$

Here d_j is the deadline. Triggering includes a one-second lookahead and uses $\geq$; candidate feasibility uses $\leq$ without that extra second. Empty K_t with active risk sets an *infeasible* flag. A latched ID is removed only when it leaves the pending queue, not when a later estimate becomes safe. The target never falls below *this tick’s* H target. A guard raise can bypass H’s cooldown and writes u_t and the current time back to H’s target and last-change state; it does

not guarantee global deadline feasibility.

While A_t is nonempty, dispatch reserves at most one concurrent offline slot when both queues have work. A running offline job leaves other free slots online-first. Otherwise, with head-of-queue slacks $s_x = d_x - t - \hat{p}_x$, online wins if $s_{\text{on}} \leq m$ and $s_{\text{on}} \leq s_{\text{off}}$; offline wins otherwise. With no online backlog, offline work may use all free slots. No running request is preempted. Target changes therefore preserve in-flight work, and later scale-out pays lifecycle delay before providing capacity.

The development search considered the registered hysteresis family and its guard variants, including 26 candidate controller configurations. Selection used the development contract and feasibility gates before the held-out inputs were read. The selected guarded configuration was identical to the preregistered H+guard arm, so a separate P_{selected} arm would have duplicated the same policy and was correctly merged into the four-arm formal matrix. This deduplication preserves the planned evidence while avoiding pseudo-replication.

7.4 Evidence protocol

The held-out study evaluates all1, all2, H, and H+guard on steady, recovery, and burst-offline windows, with three repeats per arm (36 formal runs). The GPU package is verified again on CPU using the original acceptance code and the lock-bound source snapshots. Two operator-aborted attempts are retained as history and excluded from formal statistics. All metrics are reported at run and window levels; requests are not treated as independent workloads. For Bridge, the CPU receiver independently recomputes terminal counts, lifecycle occupation, and window scenario cost from raw event streams; the sealed cell summaries are used only for consistency checks. A technically valid negative result remains in the matrix.

The gate order is fixed before looking at held-out outcomes. G_F first asks whether the two-instance reference can finish all registered work on time. M1 then compares occupied-resource error against the original model, M1b compares the lifecycle objective after deployment adaptation, and M2 checks local service fidelity on matched request IDs. M3 is a diagnostic target-agreement statistic rather than a replacement for the service and occupancy gates. Only after these model gates are interpreted do P1/P1+ test online and offline feasibility and P2 test cost. This ordering prevents an attractive cost result from rescuing a model that cannot represent the service contract, while still allowing H's failure to remain a useful ablation instead of stopping the entire matrix.

Table 2. Occupied-resource MAE (GPU seconds) by window and model. L is the equal-window lifecycle loss defined above.

Model	steady	recovery	burst	Occupied MAE	L
O	1151.97	244.08	234.48	543.51	0.09403
C	964.14	864.20	847.60	891.98	0.12162
C30	67.18	78.30	83.08	76.19	0.01353
E	66.34	66.95	70.55	67.94	0.01353
V	43.51	65.74	69.34	59.53	0.00890

8 RESULTS

We first establish the prediction-level mismatch that motivates the framework. We then run the deployment campaign—the central test of whether replay feasibility survives physical lifecycle execution—and finish with the guard control and its resource–latency boundary.

8.1 Cross-level prediction fidelity

On fixed H, E's occupied MAE is 67.94 GPU seconds versus 543.51 for O (87.50% lower). C30's improvement over C implicates startup. V reduces equal-window L by 34.24% relative to E, but E uses deployment-adapted endpoints while V retains the earlier environment snapshot. M1b is therefore not a clean same-environment causal comparison.

For local service fidelity, the fixed all1/all2 tests give a maximum window MAE of 0.064 s for C, C30, E, and V, while O reaches 3.016 s. On dynamic arms, the minimum common support across model/route projections is 122 requests and the maximum common-support MAE is 0.129 s for C, 0.112 s for C30, 0.110 s for E, and 0.112 s for V; O reaches 4.510 s. These are route-conditioned support statistics, and the complete coverage and route-confusion tables remain in the artifact.

Table 2 reports window-wise occupied-resource errors for fixed-H replay, distinct from the route-conditioned matched-ID checks.

The C-versus-O comparison reproduces the exploratory divergence: maximum matched-ID service MAE falls from 4.510 to 0.129 s (about 35-fold), while equal-window occupied-resource MAE rises from 543.51 to 891.98 GPU seconds (64.1%). These are distinct aggregation levels, not evidence that every window worsens. Changing only C's startup endpoint to C30 lowers occupied MAE to 76.19 GPU seconds, localizing a material error source within this contract.

Retrospective analysis of the frozen trajectories explains this endpoint effect and its limits (supplement). In all three windows, C/C30 targets first diverge 60 s after a common in-

Table 3. Steady-window fidelity: means over the same three physical repeats. Resource errors are GPU seconds; target disagreement is a percentage of decision ticks.

Model	Total error	Trajectory error	Target diff. (%)
C30	67.18	114.32	1.03
E	66.34	1390.50	66.73

stance start: C remains unready with higher queue/running pressure; C30 is ready. This simulator contrast is not a physical intervention on startup or cooldown. Six burst/recovery physical repeats first disagree in target with predictions at 3 s, before any window restart. Conversely, steady repeats 1 and 3 match C30 targets throughout despite state, routing, and occupation errors.

Aggregate agreement can mask temporal misalignment. In steady, E and C30 have nearly equal total occupation MAE, but E’s mean time-integrated absolute error in occupied GPU count is 1390.504 versus 114.324 GPU seconds (Table 3). Positive and negative count errors cancel in the total; their timing still matters to control. Together with the event-level counterexamples, this shows why neither startup correction nor target agreement alone establishes fidelity. The supplement retains all 36 prediction/physical pairs and 12 model/window means; this diagnosis neither changes M1 nor reselects a model. The later Bridge cells are a separate policy-contract study.

E’s equal-window target disagreement remains 26.77% against the registered 25% expectation; M3 is diagnostic, not a veto. Predicted and observed switch counts match except one burst-offline repeat (22 predicted, 24 observed). These diagnostics define the next test: freeze the replay prediction, let a learned checkpoint drive the same physical lifecycle, and compare terminal feasibility and cost from the shared ledger.

8.2 E2 policy execution: feasibility is part of the result

E2 tests whether checkpoints selected in replay remain feasible in physical execution. Each window begins with an all2 capacity anchor; the five PPO arms then run in three repeats. All 32 cells are technically valid, and the package contains 36 attempts, including four preserved technical failures. The table reports means over comparison repeats; the all2 anchor has one run per window.

Three PPO arms pass all six physical repeats: C-*s101*, C30-*r2*, and C30-ref; both all2 anchors also pass. The three feasible PPO arms have scenario costs 42.8–50.5% below all2, but this comparison is conditional on the declared ledger and synthetic API sink. C-*r2* is a decisive negative case: it averages only 4–7 on-time offline requests out of

Table 4. E2 policy execution on two A100 GPUs. P1+ requires every online and offline request to meet its deadline. Costs are registered-window scenario USD under the declared ledger. R/S denote recovery/steady. The last column is cost reduction relative to all2; † marks a failed arm and is descriptive only.

Arm	R	S	Cost R	Cost S	Cost ↓ (%)
all2	1/1	1/1	4.200	4.200	0
C- <i>s101</i>	3/3	3/3	2.357	2.449	42.8
C30- <i>r2</i>	3/3	3/3	2.073	2.084	50.5
C30-ref	3/3	3/3	2.051	2.103	50.5
C30- <i>s103</i>	0/3	0/3	1.899	2.090	52.5†
C- <i>r2</i>	0/3	0/3	2.730	2.950	32.4†

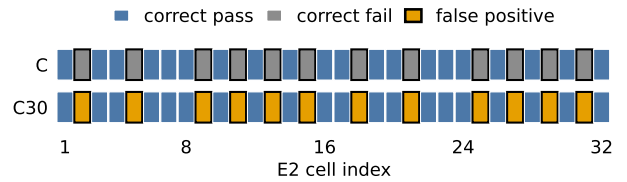


Figure 2. Replay-to-deployment feasibility for all 32 E2 cells. Blue and gray tiles are correct pass/fail directions; orange tiles are C30 false positives. Black outlines mark cells that physically fail P1+.

120 and leaves about 80 unfinished, despite lower cost. C30-*s103* is a sharper boundary: it completes the offline stream eventually, but roughly 11 of 120 offline requests miss their deadline per repeat, so its lower cost is not feasible saving. Online requests are on time in every cell; the E2 failures are therefore isolated to the offline deadline path rather than hidden by an online regression.

Predictions were frozen before physical execution. Table 5 compares each cell with its corresponding replay: C correctly predicts all 20 passes and 12 failures, whereas C30 predicts all cells feasible and misses both negative checkpoints. For each replay model, these 32 comparisons reuse 12 policy/window predictions; the physical repeats are not independent predictions. The two replay models have window-cost MAE of 0.0804 and 0.4657 scenario USD, respectively. Thus the endpoint correction that improves the earlier resource metric does not automatically improve policy-feasibility prediction.

The same reconciliation was applied to the earlier locked 38-cell Bridge campaign, which contains rule, guard, and PPO arms on its separate runtime. C matches the physical feasibility direction on 36/38 cells with 4.06% relative cost MAE; C30 matches 21/38 with 30.3% relative cost MAE. Table 6 places these results beside E2. The cost units differ by campaign (relative error for the Bridge replay and scenario USD for the deployment campaign), so the table is a compact fidelity comparison, not a pooled estimate. The

Table 5. Frozen replay versus E2 physical outcomes. Agreement counts repeated physical cells; false positives predict feasibility where P1+ fails.

Replay	Agreement	False positives	Cost MAE
C	32/32	0/12	0.0804
C30	20/32	12/12	0.4657

Table 6. Prediction-to-deployment reconciliation across the two locked campaigns. Each entry is direction agreement followed by cost MAE; the Bridge replay reports relative error and the deployment campaign reports scenario USD.

Campaign	cells	C	C30
Bridge replay	38	36/38; 4.06%	21/38; 30.3%
Deployment campaign	32	32/32; 0.0804	20/32; 0.4657

agreement pattern persists across two independently locked campaigns, while all cells and repeats remain separate.

Both training environments contain a passing and a failing checkpoint. This variation supports evaluating individual policies through the full loop, rather than ranking C and C30 training from unequal checkpoint groups. C30-ref denotes the preregistered reference checkpoint deployed as `ppo_C30_raw`; it is one of the five checkpoints, not an additional arm.

8.3 Control case study: feasibility and cost

The earlier rule-based guard remains the control case for the same contract; E2 extends that contract to frozen learned checkpoints. Table 7 summarizes the rule-control case study. All2 and all1 satisfy the registered completion target. H is technically valid but fails the P1 gate in all three recovery repeats (116/120, 113/120, and 116/120 offline on time). In that held-out study, the guard repairs this failure without changing the workload or the locked model.

H+guard’s window cost is lower than all1 by 21.96%, 48.19%, and 58.08% in steady, recovery, and burst-offline, respectively. Their equal-window mean is 42.74%, above the preregistered 25% P2 line, although steady falls below that stretch target. Including setup and cleanup gives a separate mean saving of 38.86%.

The primary ledger covers the same 2,100 s interval; the deployment ledger also charges setup and cleanup. The supplementary table retains both views, with unchanged prices and event streams.

Before execution, E’s locked recovery prediction identifies four unfinished offline requests; the three physical H repeats leave four, seven, and four unfinished, for 15 total misses. This prediction is interpreted alongside, not substituted for,

Table 7. Formal held-out results. Each arm has three repeats per window; H has 15 offline deadline misses across its three recovery repeats.

Arm	tech.	offline	online
all2	9/9	9/9 (120/120)	9/9
all1	9/9	9/9 (120/120)	9/9
H	9/9	6/9	9/9
H+guard	9/9	9/9 (120/120)	9/9

Table 8. Guard risk-active ticks and target raises, by recovery repeat.

Repeat	risk ticks	target raises	tick range	slack range
1	30	1	1982–2011	29–58
2	0	0	–	–
3	29	1	2027–2055	30–58

the observed completion evidence.

8.4 Latency–cost trade-off

The saving has a visible online latency price. The supplementary tail table reports each repeat’s online p99 rather than a pooled average. The always-on references remain near 1.6–2.6 s, while both dynamic arms reach about 15–19 s because requests can wait for the 10 s routing path and the synthetic API sink. The guard’s role is deadline protection for offline work; it does not remove this routing delay. Every observed p99 remains below the registered 60 s online deadline, so the result is an explicit SLO-margin-for-cost trade-off.

8.5 Guard activation and mechanism

The guard checks deadline risk using the visible queue, running-job ages, lifecycle delay, and concurrency-four service estimate. Equation 3 retains risk IDs until dispatch or removal from the pending queue and distinguishes risk from a target raise. For the earlier guard case study, the formal logs contain 59 risk-active ticks, all in recovery: the guard raised the target above H twice, with the remaining risk ticks maintaining the already-raised target. No risk-active tick occurred in steady or burst-offline. Per-repeat counts and slack ranges are shown in Table 8.

8.6 Price sensitivity

The cost result is conditional on the declared ledger. We recomputed all 27 fixed GPU/API price cells without changing any event stream. At the reference prices the three window savings are 21.96%, 48.19%, and 58.08%; the sign can reverse when GPU capacity is cheap and API use is ex-

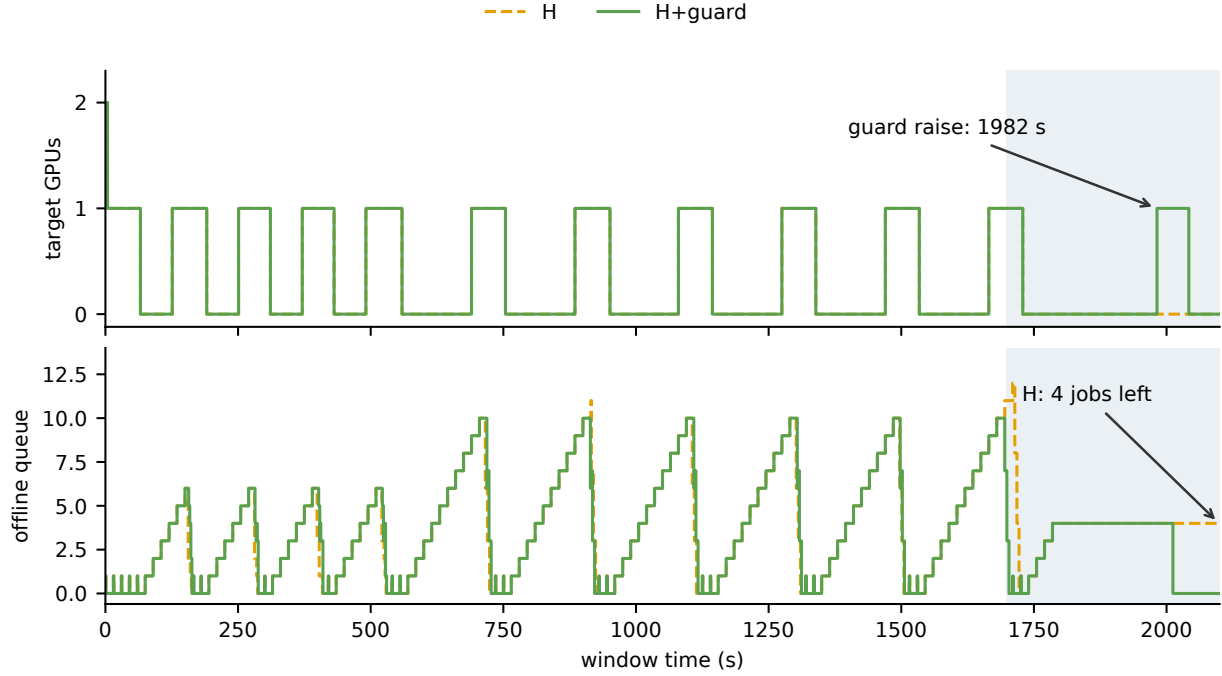


Figure 3. Recovery target and offline queue in repeat 1. The shaded final interval contains the guard raise at 1982 s; H leaves four offline jobs queued, while the guard clears them. Across all three H repeats, 15 jobs miss their deadlines.

Table 9. Price-grid savings (percent) for API multipliers 0.5/1.0/2.0. Rows are GPU multipliers 0.5/1.0/2.0; the final column is the break-even API multiplier.

Window	GPU	API .5	API 1	API 2	break-even
steady	0.5	20.05	0.17	-39.59	1.004
steady	1.0	31.90	21.96	2.07	2.104
steady	2.0	37.84	32.86	22.92	4.304
recovery	0.5	46.51	41.13	30.37	4.822
recovery	1.0	50.88	48.19	42.81	9.953
recovery	2.0	53.07	51.73	49.04	20.216
burst-offline	0.5	56.53	53.75	48.18	10.647
burst-offline	1.0	59.47	58.08	55.29	21.841
burst-offline	2.0	60.95	60.25	58.86	44.227

pensive. The only negative cell is steady at GPU multiplier 0.5 and API multiplier 2.0, where savings are -39.59% . The steady break-even API multiplier at half-price GPUs is 1.004. Recovery and burst remain positive across the tested grid, but this is a finite sensitivity analysis, not a provider billing forecast.

Separate external checks bound the rule-control results. The same-runtime baseline comparison found lower guard cost on two known Qwen windows while increasing API share and queue exposure; the Llama and load-boundary studies retained failures rather than turning them into savings. We

use these studies as scope evidence for the guard, not as additional E2 cells or pooled policy samples. Their full tables and raw identities are in the supplementary appendix.

9 DISCUSSION

Lower resource error did not imply better feasibility prediction. The C30 endpoint correction reduced fixed-H resource error, yet C predicted feasibility more accurately in both subsequent policy campaigns.

9.1 Validating predictions across levels

Better local-service fit can accompany worse closed-loop resource prediction. The startup-endpoint comparison identifies one source of this gap, while the trajectory analysis shows that aggregate occupied-resource errors can hide temporal misalignment. Validation should therefore distinguish setup from window scale-out and check states, actions, and occupied resources alongside matched-service error.

The guard shows how a controller can act on deadline risk from observed queues and frozen safety estimates. It repairs H’s held-out recovery failure, but H does not read the simulator’s startup parameters: the endpoint correction and the guard benefit have distinct causal roles.

9.2 Resource and latency trade-offs

The guard spends latency margin to reduce occupied capacity: meeting the 60-second SLO can still leave tail latency too high for an application. The steady-window cost reversal at half-price GPUs and high API prices also shows that policy choice depends on the accounting assumptions.

The same-runtime baseline comparison makes this trade-off explicit. On two known Qwen inputs, H+guard lowers cost relative to B-HPA while increasing synthetic-API share, lifecycle-event counts, and queue exposure. The comparison evaluates complete policies, including their dispatch rules. The Llama deadline miss and the failed higher-intensity Azure capacity tests show where the successful Qwen outcome stops transferring. On the first feasible tested Azure intensity, most savings occur in the final 600 seconds.

The separate controller microbenchmark measures offline `adapter.decide` calls on recovered real states; dispatch, admission, the full control loop, and GPU work are outside that measurement. Real API quality, broader scales and engines, and broader learned-policy families require further evaluation.

10 LIMITATIONS AND NEGATIVE RESULTS

The synthetic API uses a five-second timer and a routing wait; it does not generate or evaluate an answer. Consequently, the cost result is a controlled resource contract, not a claim that a commercial API can replace local serving at equal quality. GPU off is verified process and resource release, not physical power-off, energy measurement, or a cloud provider bill. The three windows are cases rather than independent workloads, and the three repeats support engineering robustness, not broad statistical generalization.

H's recovery misses show that economical cooldown decisions can leave offline work unfinished. The guard repairs the observed case without a general safety guarantee. V remains a non-isolated refitting comparison because E was adapted after deployment drift; neither the refit comparison nor the baseline comparison removes that environment caveat.

E2 covers two A100 devices, one Qwen model, two windows, and five frozen checkpoints. It establishes neither heterogeneous-device scalability nor a general ranking of PPO training environments or PPO versus the guard. The 38-cell Bridge campaign uses a separate runtime and is not pooled with E2. Llama adds one similarly sized family and two repeats, including a guarded deadline failure; broader model scales, quantization, and engines remain untested. The same-runtime baseline compares two complete policies on known Qwen inputs, so it neither adds held-out workloads nor isolates the guard inside the other controllers. Can-

didate families and workloads were locked before replay, with no window replacement. Missing session identities and regular offline pressure limit population-level inference. CPU replay covers accepted records; the archive's missing root-level convenience snapshot remains a reproducibility warning.

11 CONCLUSION

FidelityLoop provides a replayable contract for connecting elastic-serving policy decisions to lifecycle execution and resource/cost accounting. It shows why predictions need validation across levels: local-service fit can improve while resource prediction worsens, and accurate total occupation can hide large trajectory errors. Startup-endpoint isolation and event-level counterexamples explain where these checks diverge. Lifecycle states, control decisions, and resource use should therefore be validated together.

The guard case study quantifies deadline protection and its resource, routing, and cost trade-offs. E2 extends the same contract to frozen PPO checkpoints: some checkpoints reduce scenario cost while preserving every deadline, whereas the negative checkpoints reveal why cost must be conditioned on feasibility. Under the studied synthetic-API setting, the practical lesson is to verify deadline feasibility before comparing resource cost.

REFERENCES

- Agrawal, A., Kedia, N., Mohan, J., Panwar, A., Kwatra, N., Gulavani, B. S., Ramjee, R., and Tumanov, A. Vidur: A Large-Scale Simulation Framework for LLM Inference. In *Proceedings of Machine Learning and Systems*, volume 6, pp. 351–366, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/b74a8de47d2b3c928360e0a011f48351-Abstract-Conference.html.
- Crankshaw, D., Sela, G.-E., Zumar, C., Mo, X., Gonzalez, J. E., Stoica, I., and Tumanov, A. InferLine: ML Prediction Pipeline Provisioning and Management for Tight Latency Objectives, 2018. URL <https://arxiv.org/abs/1812.01776>.
- Fu, Y., Xue, L., Huang, Y., Brabete, A.-O., Ustiugov, D., Patel, Y., and Mai, L. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models, 2024. URL <https://arxiv.org/abs/2401.14351>.
- Gujarati, A., Karimi, R., Alzayat, S., Hao, W., Kaufmann, A., Vigfusson, Y., and Mace, J. Serving DNNs like Clockwork: Performance Predictability from the Bottom Up, 2020. URL <https://arxiv.org/abs/2006.02464>.

-
- Kabakibo, H. S., Trivedi, A., and Wang, L. Breaking the Ice: Analyzing Cold Start Latency in vLLM. In *Proceedings of Machine Learning and Systems*, volume 8, pp. 1187–1205, 2026. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/29416b66c2149872b9d1415a3fd2c5e0-Abstract-Conference.html.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient Memory Management for Large Language Model Serving with PagedAttention, 2023. URL <https://arxiv.org/abs/2309.06180>.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal Policy Optimization Algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.
- Sun, B., Huang, Z., Zhao, H., Xiao, W., Zhang, X., Li, Y., and Lin, W. Llumix: Dynamic Scheduling for Large Language Model Serving, 2024. URL <https://arxiv.org/abs/2406.03243>.
- Wang, Y., Chen, Y., Li, Z., Kang, X., Tang, Z., He, X., Guo, R., Wang, X., Wang, Q., Zhou, A. C., and Chu, X. BurstGPT: A Real-world Workload Dataset to Optimize LLM Serving Systems, 2024. URL <https://arxiv.org/abs/2401.17644v3>.
- Yang, M., Zhang, Z., Wu, M., Yan, J., Sun, H., and Chang, L.-W. Charon: A Unified and Fine-Grained Simulator for Large-Scale LLM Training and Inference. In *Proceedings of Machine Learning and Systems*, volume 8, pp. 94–107, 2026. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/dbc8ce0fdcd55172d73fb05dbae07fc-Abstract-Conference.html.
- Zhang, C., Yu, M., Wang, W., and Yan, F. MArk: Exploiting Cloud Services for Cost-Effective, SLO-Aware Machine Learning Inference Serving. In *2019 USENIX Annual Technical Conference*, pp. 1049–1062, 2019. URL <https://www.usenix.org/conference/atc19/presentation/zhang-chengliang>.
- Zhang, G., Guo, W., Tan, Z., Guan, Q., and Jiang, H. KISS: A GPU-Aware Kubernetes Inference Simulator with RL-Based Auto-Scaling, 2025a. URL <https://arxiv.org/abs/2507.07932>.
- Zhang, G., Vippagunta, S., Nandagopal, R., Raman, S., Xu, J., Pfeiffer, M., Chatterjee, S., Tan, Z., Guo, W., and Jiang, H. AAPA: An Archetype-Aware Predictive Autoscaler with Uncertainty Quantification for Serverless Workloads on Kubernetes, 2025b. URL <https://arxiv.org/abs/2507.05653>.
- Zhang, H., Tang, Y., Khandelwal, A., and Stoica, I. SHEPHERD: Serving DNNs in the Wild. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation*, pp. 787–808, 2023. URL <https://www.usenix.org/conference/nsdi23/presentation/zhang-hong>.